Execution Report

Introduction

In this execution report I'll be sending some trade orders to an API to see how much the live execution actually differs from the backtests I've conducted. I'll be recording several metrics, most importantly the latency and slippage of the trades being made. For this test, I used the Oanda API and a playlist of Youtube videos made by the user Bluefever Software as well as his Github repository, which I will link in the sources below. The majority of the code comes directly from that Github repository, however I made some changes to the execution logic so I could send orders according to a long and a short strategy that I've backtested and also extract the data pertaining to latency and slippage.

Latency and Slippage Measures

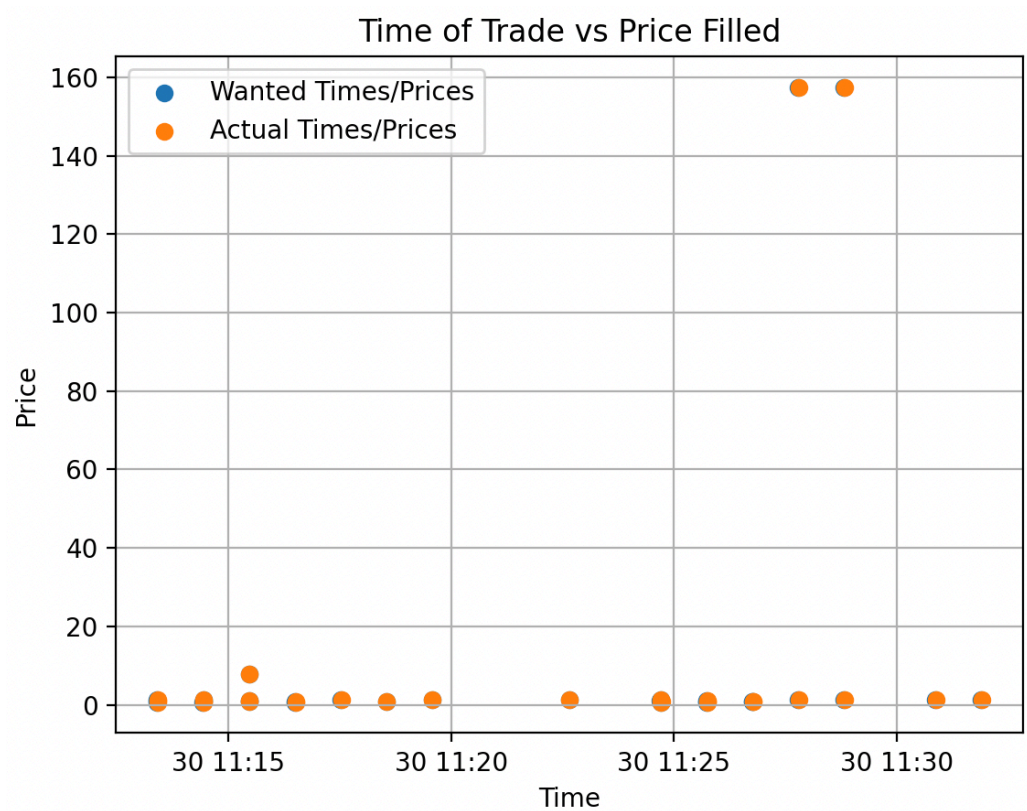
The latency and slippage are measured as follows: Whenever a signal is generated from the long or short strategy, the trading bot will record the timestamp of when the signal was generated, that is, when the decision was made to go long or short on the currency pair. When the order is filled, the API will send, among many other things, an exact timestamp of when the order was filled. These two timestamps are then written to the log. Similarly, when the decision to trade has been made, the bot will record the price at that timestamp as the price to be filled (because that is the assumption that was made during the backtesting phase), as well as the actual fill price returned to the bot by the API. These four pieces of information are all that is needed to calculate the latency (actual - intended timestamp) and slippage (actual - intended price). The bot also records other necessary information like the currency traded, whether the trade was long or short, and the trade ID number.

Visualizations

The two tests here were performed at different times during the day/early morning because of two reasons. The first being that the bot was being tested during production and went live in the night time, and the second being that the bot was ready and executed trades for the same amount of time after confirming that it wouldn't crash. The datasets both have size 30, meaning that there were 15 trades in each run, each with 2 records (desired time/price and actual time/price).

Below is a plot for the second dataset. This was done during the morning hours EST when the market has been open for a while and there is high volume in many of the currency pairs. The blue dots represent the times that the orders were sent and the prices at which they were filled, while the orange dots represent the times that the orders were actually filled and at which prices. According to the data, there was actually no slippage on any of the short or long trades,

which is quite surprising. Hence, each orange and blue dots pair is at the same height. However, there was slight latency on every trade because of the fact that new bars are being passed into the pre-existing (vectorbt) code to generate a signal instead of being efficiently recomputed using old bars. This dataset has an average latency of 0.701341293 seconds, with a standard deviation of 0.236573 seconds.



On the nighttime/early morning dataset we get the following plot.

Clearly, the orange and blue dots are effectively on top of each other again, meaning that there wasn't a lot of latency or slippage. The exact numbers are as follows: The average amount of slippage was 2.7e-5 seconds, with variance 3.097586 seconds and the average latency was 0.7593817 seconds with a standard deviation of 0.50616 seconds. Interestingly, among these trades, the only ones with any slippage were the short trades. I'm not sure why this would be the case, and this seems to suggest that there is more latency in short trading, but this is not the case either. The average latency for the short trades was almost 12 milliseconds faster (0.7498315 secs vs 0.761769 secs). Another possible explanation could be that certain currencies were less liquid during the time when the bot was trading to generate this dataset.

References

1. https://www.youtube.com/watch?v=zKk2iuuNJO0&list=PLZ1QII7yudbecO6a-zAI6cuGP1LLnmW8e&index=1

2. https://github.com/bluefeversoft/forex_python/tree/master/Part75_Trading_Bot

3. https://github.com/avasisht1/AQFAAT_Backest